AgentShield-Crypto: Evaluating Cryptographic Authorization, Durable Replay Protection, and Effect Safety for Tool-Using AI Agents

Saiful Islam Tanvir
AI and Cybersecurity Researcher
saifislamtanvir07@gmail.com

***Abstract*—Tool-using AI agents turn a language-model proposal into a real side effect, such as placing an order or calling a privileged service. Checking that the proposal is well formed is not enough. The system also has to establish who authorized it, under which policy and for which service, whether that authorization is still valid, and whether retries or concurrent workers can apply it more than once. We present AGENTSHIELD-CRYPTO, an executable reference design for a single action-authorization path, together with a controlled evaluation of each layer. The gate combines strict wire parsing, Ed25519 signatures with algorithm pinning, key-lifecycle checks, mandate expiry, audience and policy-version binding, proposal-digest binding, a durable SQLite replay ledger, and an idempotent downstream exchange. We ablate nine designs against 1,020 constructed attack requests each, plus a valid control and an in-policy semantic-manipulation family. The full design executes 0 of 1,020 attacks (95% Wilson upper bound 0.38%) and accepts all 60 valid requests, while an intentionally weak variant that trusts the envelope's algorithm label executes 60 forgeries. Across 30 trials with eight concurrent processes, the durable ledger yields exactly one effect per trial against eight for a process-local replay set. A crash-injection harness and an exhaustive finite-state model show that a local ledger alone cannot give exactly-once effects: safety needs an idempotent receiver or reconciliation before retry. Storage settings, not the signature scheme, dominate latency. Cryptographic authorization is necessary but does not make an in-policy action safe, and we state this residual risk explicitly. All results are synthetic and local; we make no production or universal claims.**

***Index Terms*—AI agents, tool use, authorization, digital signatures, replay protection, idempotency, algorithm confusion, crash recovery, security evaluation.**

I. INTRODUCTION

AI agents increasingly act through tools instead of only producing text. A model may recommend a purchase, edit a record, trigger a deployment, or call an internal API. Once that happens, a plausible-looking model output becomes a consequential side effect, and the security boundary moves. The application can no longer ask only whether a response looks reasonable. It has to decide whether this particular action was authorized, by whom, under what rules, and whether it is safe to execute right now and exactly once.

A common implementation mistake is to treat a tool call as a trusted continuation of the model conversation. In practice the proposal can be malformed, altered after approval, replayed after a restart, issued under a key that has since been revoked, or perfectly valid at the protocol level while still violating business policy. Concurrency adds a second family of problems. Even when a request is authorized only once in principle, an ambiguous timeout leaves the caller unsure whether the external operation already happened, and a blind retry may apply it twice.

This paper looks at those problems through AGENTSHIELD-CRYPTO, a Python prototype that places a deterministic authorization gate between an agent's proposal and a simulated exchange. We chose a deliberately small action vocabulary (a token, a buy or sell action, and an integer amount) so that every check can be inspected and every failure can be reproduced. Our research question is narrow on purpose:

> *How much protection do layered cryptographic authorization and durable effect-handling controls provide against a defined set of request-level attacks, and which risks remain when all of them are in place?*

We are not trying to show that a language model can recognize harmful intent, and we do not claim a new signature primitive or a general fix for prompt injection. The contribution is an engineering and evaluation one, and we have tried to keep each claim as strong as the evidence and no stronger.

Contributions.

1) *A reference design with a precise acceptance predicate.* We separate a signed mandate from the untrusted proposal and bind the two with a proposal digest, and we write the full gate as one predicate whose order matches the implementation (Section **??**).
2) *A controlled ablation over nine designs.* Each design faces 1,020 attack requests, a 60-request valid control, and an in-policy semantic family that we keep as residual risk. The aggregate drops can be attributed, step by step, to specific attack families (Table **??**).
3) *A formal and empirical account of algorithm confusion.* We show why letting the envelope pick the verification algorithm lets anyone who can read a verifier's key forge a MAC, and we confirm it experimentally (Section **??**).
4) *An evaluation that separates request uniqueness from effect uniqueness.* A 200-scenario crash harness, an exhaustive model check of two racing gates, and a 30-trial multi-process race together show when a local ledger is enough and when it is not.
5) *An operational measurement.* We quantify the revocation window created by key caching and show that SQLite

journaling and synchronization, rather than signature verification, dominate end-to-end latency.

Section **??** places the work in context. Sections **??** and **??** give the model and design, Section **??** states the properties we test, and Sections **??** and **??** present the method and results. Sections **??**–**??** discuss limits and conclude.

II. Background and Related Work

A. Signed authorization artifacts and their pitfalls

Digital signatures and message authentication codes are the standard tools for integrity and origin authentication. Ed25519 offers a fast public-key signature with a compact interface [**?**], [**?**], and HMAC is the usual symmetric alternative [**?**]. Our work introduces neither. What interests us is how these primitives behave once they sit inside a larger authorization path.

Token formats show how fragile that path can be. JSON Web Tokens [**?**] let the token header name the signing algorithm, and the best-current-practice document for JWTs [**?**] warns that a verifier which honors the header can be tricked into using a public key as an HMAC secret. Our D7n ablation recreates this class of bug in a small, controlled setting. Capability-style tokens such as macaroons [**?**] attach contextual caveats to delegated authority. Our mandate is much simpler, but it follows the same instinct of making the conditions of use (audience, policy version, lifetime, exact action) part of the signed object.

B. Canonicalization and parser disagreement

A signature is only as unambiguous as the bytes it covers. JSON allows formatting variation, and common parsers disagree on duplicate member names. RFC 8785 defines a canonicalization scheme for JSON [**?**]. Our prototype rejects duplicate keys, floats, and non-finite constants and serializes with sorted keys and ASCII escaping. It is *not* claimed to be JCS-compliant; a production system should adopt a standard profile and publish cross-language test vectors.

C. Freshness, replay, and idempotence

Replay and freshness have a long history in authentication protocols [**?**], [**?**]. A signature does not say the message is new, so an implementation needs expiry, a nonce, or stored state. For effects, the relevant literature is about the limits of local state. A local database and a remote service do not share a transaction, and what a caller can do after a lost response depends on whether the receiver is idempotent [**?**], [**?**]. We narrow this to a SQLite ledger and a simulated exchange and make no claim about a new commit protocol. For checking the recovery logic we use exhaustive state exploration in the spirit of explicit-state model checkers [**?**].

D. Security of LLM agents

Indirect prompt injection showed that untrusted content retrieved by an agent can steer its tool use [**?**]. Benchmarks such as AgentDojo [**?**] and InjecAgent [**?**] measure how often agents follow injected instructions, and system-level defenses such as CaMeL [**?**] separate control flow from untrusted data. Community guidance treats excessive agency as an application-level concern [**?**]. These works mostly ask whether the *agent* can be manipulated. We ask a complementary question about the *executor*: given that a proposal exists, can the system refuse anything it was not authorized to do, and can it avoid doing an authorized thing twice? Our scripted-agent experiment (Section **??**) is explicitly not a model evaluation. It isolates the gate by assuming the worst case, an agent that always follows the injected goal.

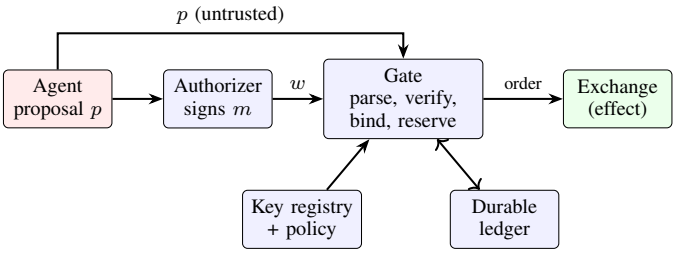


Fig. 1. Authorization path. The gate trusts only the registry, the policy, and its own ledger. The proposal and the wire envelope are untrusted inputs.

III. System and Threat Model

A. Entities and flow

We distinguish five entities. The *proposer* (a language model, planner, or application component) emits a candidate action p. The *authorizer* decides whether p may be attempted and signs a mandate m. The *key registry* maps a key identifier to trusted verification material, an algorithm, and lifecycle metadata. The *gate* validates (w, p) and coordinates execution. The *exchange* is the downstream service that creates the externally visible effect (Fig. **??**). The model output is never a credential: the gate treats both the wire envelope w and the proposal p as untrusted.

A proposal is $p = (token, action, amount)$ with the amount an integer number of cents. A mandate contains a request identifier, audience, policy version, issue and expiry times, a digest of the proposal, and a decision object $(approved, action, token, amount)$. The decision repeats the action fields on purpose, so the gate can compare them to the proposal instead of assuming a valid signature binds every application field.

B. Adversary

The test adversary can submit arbitrary bytes and malformed JSON, use unknown key identifiers, alter signatures and proposals, replay old or already-consumed requests (including after a restart), present expired mandates, present validly signed requests that violate policy, and exploit a deliberately weakened verifier that trusts the envelope's algorithm label. It may submit concurrently through many processes and retry after simulated failures. For the full design the adversary does *not* hold a signing key. One family, *verifier-material leakage*, gives the adversary the verifier's key bytes to illustrate the difference between a public verification key and a shared secret.

C. Trust assumptions and out-of-scope threats

We assume the policy and key registry are operated by a trusted party, the signing key is uncompromised in the main experiments, the gate's SQLite files are available and correctly configured, and the exchange implements the stated idempotency behavior when that option is on. We do *not* model a malicious operating system, a compromised database administrator, rollback of an entire storage volume, Byzantine replicas, cross-region partitions, a compromised authorizer, or production key management. Test keys are generated fresh per run and are never derived from any public label.

D. Objectives

We evaluate six objectives. **O1** (integrity): altered or unauthenticated envelopes must not verify. **O2** (context): audience, policy version, key status, and time window must match. **O3** (binding): the submitted proposal must be the one the mandate authorizes. **O4** (replay): a request identifier must not produce a second accepted dispatch, even across restarts. **O5** (effect deduplication): retries and concurrent submissions must not create duplicate effects under stated idempotency assumptions. **O6** (explicit residual risk): a valid, in-policy request may still encode an unintended goal, and the gate is not a semantic safety oracle.

IV. Design

A. Strict parsing and signing input

The parser rejects oversized messages (above 8,192 bytes), duplicate keys, floating-point numbers, and non-finite constants. Duplicate-key rejection matters because parsers disagree on whether the first or last value wins, and the gap between two components' readings of one message is a classic attack surface. After parsing, a schema check validates the version, algorithm label, key identifier, signature encoding (lowercase hexadecimal), every mandate field, the action vocabulary, the integer type of amounts, timestamps, and identifier formats. Let $\mathcal{C}(\cdot)$ denote the prototype's canonical serialization (sorted keys, compact separators, ASCII escaping, no `NaN`), and let $D = \texttt{AgentShield-v1} \,\|\, \texttt{0x00}$ be a domain-separation prefix. The signing input is

$$\mathsf{sin} \;=\; D \,\|\, \mathcal{C}\big(\{v,\ alg,\ kid,\ m\}\big), \tag{1}$$

which excludes only the signature field. Because the signature covers the canonical bytes produced by the protocol, a receiver cannot choose a different re-serialization. The proposal digest is $H(\mathcal{C}(p))$ with H = SHA-256, compared in constant time.

B. Envelope, algorithm pinning, and key registry

The full design verifies Ed25519 signatures [**?**]. The verification key and its *configured algorithm* come from a trusted registry row, not from the envelope. The gate rejects any envelope whose `alg` differs from the row's algorithm. HMAC-SHA-256 [**?**] is kept in earlier designs as a comparison. The registry stores validity bounds $[t_{nb}, t_{na})$ and an optional revocation time t_{rv}, and a key is *active* at time t iff $t_{nb} \le t < t_{na}$ and (t_{rv} is null or $t_{rv} > t$). Optional caching of registry rows with a time-to-live trades read cost for revocation latency (Section **??**).

C. Acceptance predicate

Let w be the wire, p the proposal, t the evaluation time, K the resolved key record, L the durable ledger, $\delta = 5\,\mathrm{s}$ the clock-skew allowance, and $T_{\max} = 600\,\mathrm{s}$ the maximum mandate lifetime. The full design accepts iff

$$\begin{aligned}\mathrm{Accept}(w,p,t) \equiv\ & \mathrm{Parse}(w) \wedge \mathrm{Schema}(w) \wedge \mathrm{Shape}(p)\\ & \wedge\ \mathit{approved}(m) \wedge \mathrm{Policy}(p)\\ & \wedge\ \mathrm{Known}(K) \wedge \mathrm{Active}(K,t)\\ & \wedge\ \mathit{alg}(w) = \mathit{alg}(K)\\ & \wedge\ \mathrm{Verify}_K(\mathsf{sin},\sigma) \wedge \mathrm{Fresh}(m,t)\\ & \wedge\ \mathit{aud}(m) = \mathit{aud}_{\mathrm{local}}\\ & \wedge\ \mathit{ver}(m) \ge \mathit{ver}_{\mathrm{policy}}\\ & \wedge\ \mathrm{Bind}(m,p) \wedge \mathrm{ReserveNew}(L,\mathit{rid}),\end{aligned} \tag{2}$$

where

$$\begin{aligned}\mathrm{Fresh}(m,t) \equiv\ & m.\mathit{iat} \le t + \delta\ \wedge\ t < m.\mathit{exp}\\ & \wedge\ m.\mathit{exp} - m.\mathit{iat} \le T_{\max},\end{aligned} \tag{3}$$

$$\begin{aligned}\mathrm{Bind}(m,p) \equiv\ & (d.\mathit{tok}, d.\mathit{act}, d.\mathit{amt}) = p\\ & \wedge\ m.\mathit{ph} = H(\mathcal{C}(p)).\end{aligned} \tag{4}$$

Two details differ from a casual reading and are easy to get wrong. The policy-version rule is a *minimum* ($\ge$), so a newer mandate is fine and a downgraded one is not, and the expiry bound is strict ($t < m.\mathit{exp}$). The order is security-relevant: parsing precedes any use of nested fields, the key and signature checks precede trusting the mandate, and the reservation, the only step with a persistent effect on the gate, comes last so that rejected requests leave no ledger row.

D. Durable replay ledger

A process-local set is useless when several workers hold independent memory. The ledger therefore reserves a request identifier inside a SQLite `BEGIN IMMEDIATE` transaction, so competing writers serialize before the insert decision, and the primary key on the identifier is the cross-process exclusion point. The row stores a digest of the *received wire bytes* and a state in {RESERVED, DISPATCHING, AMBIGUOUS, CONFIRMED}. Re-presenting the same identifier with the same wire digest is a duplicate; a different digest is a conflict, not a retry. Conflict rows are never overwritten.

E. Effect handling and recovery

A reservation cannot atomically commit an effect in an unrelated service. Between applying the effect and recording confirmation there is an ambiguous window, and a crash or lost response there leaves the gate unsure whether the effect exists. We test two remedies. In the first, the exchange is *idempotent*: it stores the idempotency key in the same transaction as the effect, and the gate uses the request identifier as that key. In the second, recovery is *reconcile-first*: before retrying a row in DISPATCHING or AMBIGUOUS, the recovery routine asks the exchange whether an effect with that key already exists. The distinction is central to the paper: the ledger gives

Algorithm 1 Gate processing of one request

Require: wire w, proposal p, time t

1: $e \leftarrow$ StrictParse(w); check schema; check Shape(p)
2: **if** $\neg e.m.decision.approved$ **or** $\neg$Policy(p) **then**
3: **return** reject
4: **end if**
5: $K \leftarrow$ Registry($e.key_id$)
6: **if** $K = \perp$ **or** $\neg$Active(K, t) **or** $e.alg \neq K.alg$ **then**
7: **return** reject
8: **end if**
9: **if** $\neg \text{Verify}_K(D \| \mathcal{C}(e \setminus \sigma), \sigma)$ **then**
10: **return** reject
11: **end if**
12: **if** $\neg$Fresh $\vee$ $\neg$Audience $\vee$ $\neg$Version $\vee$ $\neg$Bind **then**
13: **return** reject
14: **end if**
15: $s \leftarrow$ Reserve(rid, SHA256(w)) {atomic}
16: **if** $s =$ Dup **then**
17: **return** reject(replay)
18: **end if**
19: **if** $s =$ Conflict **then**
20: **return** reject(id conflict)
21: **end if**
22: SetState(rid, Dispatching); submit to exchange with key rid
23: **if** response received **then**
24: SetState(rid, Confirmed);
25: **return** accept
26: **else**
27: SetState(rid, Ambiguous);
28: **return** ambiguous
29: **end if**

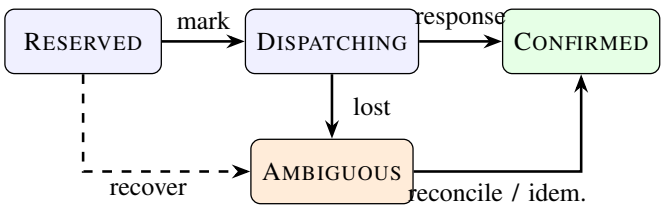


Fig. 2. Ledger states for one request identifier. The AMBIGUOUS state is where effect uniqueness is decided: the exit from it must either query the exchange or rely on its idempotency key.

request uniqueness, and *effect* uniqueness needs one of the two remedies.

V. Properties and Algorithm Confusion

A. Safety properties

We interpret the experiments through five engineering properties of the implemented state machine under the assumptions of Section **??**. They are not cryptographic proofs of a deployed system.

P1 (invalid-message non-execution). A request rejected by parsing, authentication, context, binding, or policy creates no exchange effect. **P2 (single reservation).** For a given identifier, at most one concurrent process receives the new-reservation result. **P3 (conflict detection).** Reusing an identifier with a different wire digest is never treated as a retry. **P4 (effect uniqueness under idempotency).** If the exchange records the idempotency key atomically with the effect, repeated submissions yield at most one effect. **P5 (recovery progress).** In the safe configurations, a request left in an intermediate state can be finished without a duplicate effect and always reaches a confirmed state.

Why they hold, and where. P1 follows from the ordering in Eq. (**??**): the only code path that reaches the exchange is after every check. P2 follows from the primary-key constraint plus the immediate write lock, since two inserts with the same key cannot both succeed. P4 holds because the idempotency-key insert and the effect insert share a transaction. P1, P2, P4, and P5 are tested empirically below. P3 holds by construction (the stored digest is compared on every duplicate), but our recorded run has no dedicated conflicting-identifier test, so we report it as a design property and not as a measured one.

B. Why a local ledger is not enough

The crash results are not an accident of our implementation. Consider a gate that has sent the order and then loses the response. If the exchange offers neither an idempotency key nor a way to query whether the key was applied, the two worlds "the order was applied" and "the order was never received" produce the same local observation. A retry policy then has only two choices: retry, which duplicates the effect in the first world, or abandon, which loses the effect in the second. No amount of local state removes this ambiguity, because the missing information lives in the receiver. This is the usual indistinguishability argument behind idempotent-receiver designs [**?**], and it is why our recovery experiment varies the *receiver* and not only the ledger. The practical consequence, which the model check confirms exhaustively for two gates, is that exactly-once behavior is a property of the pair (gate, exchange).

C. Algorithm confusion

Suppose a verifier computes the algorithm from the envelope, so that $alg(w) = \texttt{hmac}$ selects HMAC with the registry row's key bytes μ_K. For an Ed25519 key, μ_K is the *public* key. Define the forgery

$$\sigma^* \;=\; \text{HMAC}_{\mu_K}\big(D \,\|\, \mathcal{C}\{v, \texttt{hmac}, kid, m^*\}\big) \qquad (5)$$

for any chosen mandate m^*. A verifier that takes the algorithm from the envelope recomputes exactly σ^* and accepts. Anyone who can read μ_K can therefore produce an accepted mandate without ever touching the private key. Pinning the algorithm to the registry row (the rejection in Algorithm **??**, line 4) removes the attack, because the envelope's label is only compared, never obeyed. The same reasoning explains why a shared HMAC secret is fragile when leaked: here the verifier *is* the signer. In our harness the verifier-leak family gives the adversary μ_K and builds σ^* directly.

VI. Experimental Methodology

A. Implementation and environment

The prototype is Python with SQLite for the registry, ledger, and exchange. The recorded full run used Python 3.13.15, Linux 6.18.48, NumPy 2.1.3, pandas 2.3.3, `cryptography` 50.0.1, SQLite 3.45.1, and four CPUs. Workload generation is seeded (1337). Cryptographic keys are fresh random bytes per run, so key material is not reproducible by design, only the workload is. The recorded run used the full sizes (60 cases per family, 2,000 measured requests, five repeats), not the smoke-test mode. Every result file is hashed with SHA-256 into a run manifest, and the notebook asserts 19 automatic claim checks (all passed) so a silent regression would be visible.

B. Designs and attack families

Table **??** lists the nine cumulative designs. The corpus has 19 families: one valid control, 17 attack families, and one residual-risk family (in-policy semantic manipulation). Each family has 60 generated cases, giving $17 \times 60 = 1{,}020$ attack requests per design. The 17 attack families are unsigned forgery, signature bit flip, amount tampering, approval tampering, proposal swap, exact replay, replay after restart, unknown key, revoked key, expired key, not-yet-valid key, expired mandate, audience mismatch, policy-version downgrade, signed over-limit order, verifier-material leakage, and malformed wire (half truncated, half with a duplicated key). Replay-after-restart reopens the gate on the same files so that persistence, not only a process-local set, is tested. We report counts and rates as primary evidence and a 95% Wilson interval as a supplementary description of sampling variation inside the generated corpus.

C. Statistical reporting

We report counts first and intervals second. For the 95% Wilson score interval [**?**], 0 of 1,020 gives an upper bound of 0.38%, and 0 of 60 gives 6.0%, so a single family of 60 cases cannot rule out a failure rate of a few percent. The 60 verifier-leak executions in D6 and D7n out of 1,020 give an interval of 4.6% to 7.5%, and 60 of 60 accepted controls gives a lower bound of 94.0%. These intervals describe sampling variation inside a generated corpus. They are not estimates of how an unknown attacker would fare, and they ignore attack classes we did not think of.

D. Crash, model-check, and race experiments

The crash harness injects abrupt process termination (`os._exit(137)` in a forked child) after reservation, after the dispatch mark, after the effect is applied, and after confirmation, and a fifth case where the effect lands but the response is lost. It crosses idempotent versus non-idempotent exchange with blind retry versus reconcile-first recovery: 4×5 cells with ten repetitions, 200 scenarios, each followed by recovery and a client retry of the same wire. For the model check, we encode two gate processes and a shared ledger and exchange as a transition system with up to two crashes and two lost responses, enumerate every reachable state breadth-first, and test three invariants: **I1**, the exchange never holds more than one effect; **I2**, an effect never exists while the ledger is still before the dispatch mark; and **I3**, from every reachable state a state with state CONFIRMED and exactly one effect is still reachable (recovery progress). For the race, eight forked processes pass a barrier and submit one identical signed request, 30 trials per design.

E. Reproducibility

Every experiment writes a CSV (attack matrix, crash outcomes, model check, race, revocation, component and end-to-end timings, agent scenario, claim checks), and a manifest records the seed, workload sizes, library versions, and a SHA-256 digest for each file, so a later reader can tell whether a reported number matches a given run. The notebook should be restarted and run end to end with the smoke-test switch off before any number is quoted, and any code change after a reported run requires regenerating the affected results. Only synthetic keys and data are used.

F. Revocation, performance, and agent scenarios

For revocation we warm the registry cache, revoke the key one second later, and probe at 1, 9, and 119 s after revocation with cache TTL of 0, 5, and 60 s. Performance uses six end-to-end configurations (five repeats of 2,000 measured requests after 30 warm-up requests, with a bootstrap interval for the pooled median) and component microbenchmarks (3,000 iterations for crypto and parsing, 600 for ledger writes). All timings are sequential and local. For the agent scenario we use a deterministic worst-case agent that always follows the target of the scenario (benign, over-limit, off-allowlist, forged approval, in-policy manipulation) across eight benign news items and three injection templates, and compare four defense levels: L0 no checks, L1 policy only, L2 policy plus an in-band approval flag the sender can set, and L3 the full gate. This isolates the executor and does not measure how often a real model would obey an injection; an optional local-model path exists in the notebook but was not enabled in the recorded run.

VII. Results

A. Attack matrix and ablation trajectory

Table **??** and Fig. **??** show attack requests that produced an effect. The full design executes 0 of 1,020 (95% Wilson upper bound 0.38%) and accepts all 60 valid controls. Every design accepts all 60 valid controls and every design executes all 60 in-policy semantic requests. The residual family is therefore excluded from the attack denominator on purpose, not hidden.

Because each step in the ablation adds mostly one control, the drops in Table **??** can be attributed to families (Table **??**). The attribution is derived from the count differences together with how each family is built; the per-cell values are in the released matrix file. It shows that no single control is sufficient. Policy alone leaves 900 executions because it authenticates nothing. Signatures remove the largest block (300) but leave replay, lifecycle, audience, and binding attacks open. Durable replay state and key-lifecycle and binding checks together

TABLE I
CORPUS FAMILIES, HOW EACH CASE IS CONSTRUCTED, AND THE PROPERTY IT PROBES. EVERY FAMILY HAS 60 GENERATED CASES.

Family	Construction	Kind	Probes
valid	Correctly signed, in-policy order	control	acceptance of legitimate use
unsigned forgery	Well-formed envelope with an empty signature	attack	O1
signature bit flip	Valid wire with the first signature character changed	attack	O1
amount tampering	Amount doubled and digest recomputed, original signature kept	attack	O1, O3
approval tampering	A signed rejection edited to read approved	attack	O1
proposal swap	Valid mandate, but a different amount is submitted	attack	O3
exact replay	Same wire submitted twice	attack	O4
replay after restart	Same wire, gate reopened on the same files	attack	O4 (persistence)
unknown key	Signed by a key absent from the registry	attack	O2
revoked / expired / future key	Signed by a key in each lifecycle state (three families)	attack	O2
expired mandate	Issue and expiry both in the past	attack	O2
audience mismatch	Mandate addressed to another venue	attack	O2
policy downgrade	Policy version below the current minimum	attack	O2
signed over-limit	Signed order above the amount cap	attack	policy
verifier leak	Attacker holds the verifier's key bytes and builds an HMAC	attack	algorithm pinning
malformed wire	Half truncated, half with a duplicated `approved` key	attack	parser
in-policy semantic	Valid, signed, in-policy order that is simply unwise	residual	O6

TABLE II
CUMULATIVE DESIGNS. S = STRICT PARSE; P = POLICY; E = EXPIRY; R = REPLAY STATE (M = MEMORY, D = DURABLE); L = KEY LIFECYCLE; B = AUDIENCE, VERSION AND PROPOSAL BINDING; I = IDEMPOTENT EXCHANGE.

Design	S	P	Auth	E	R	L	B	I
D0 none	–	–	–	–	–	–	–	–
D1 policy	✓	✓	–	–	–	–	–	–
D2 +hmac	✓	✓	HMAC	–	–	–	–	–
D3 +expiry	✓	✓	HMAC	✓	–	–	–	–
D4 +mem replay	✓	✓	HMAC	✓	M	–	–	–
D5 +durable	✓	✓	HMAC	✓	D	–	–	–
D6 +life+bind	✓	✓	HMAC	✓	D	✓	✓	–
D7 full	✓	✓	Ed25519	✓	D	✓	✓	✓
D7n naive alg	✓	✓	Ed25519	✓	D	✓	✓	✓

D7n is D7 with algorithm pinning disabled: the envelope label chooses the verifier.

TABLE III
ATTACK REQUESTS EXECUTED PER DESIGN (LOWER IS BETTER; $n = 1{,}020$). VALID CONTROLS ACCEPTED: 60/60 IN EVERY DESIGN. RESIDUAL IN-POLICY FAMILY EXECUTED: 60/60 IN EVERY DESIGN.

Design	Executed	Rate
D0 none	990	97.1%
D1 policy	900	88.2%
D2 +hmac	600	58.8%
D3 +expiry	540	52.9%
D4 +mem replay	480	47.1%
D5 +durable	420	41.2%
D6 +life+bind	60	5.9%
D7 full	**0**	**0.0%**
D7n naive alg	60	5.9%

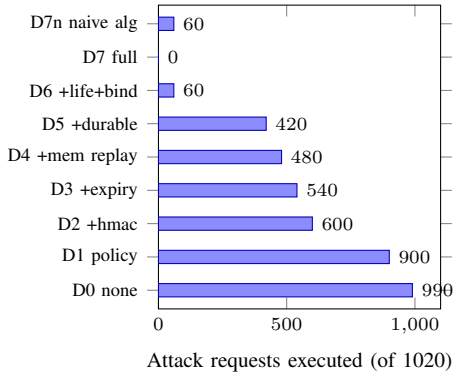


Fig. 3. Executed attacks by design. The only residual in D6 and D7n is verifier-material leakage, which succeeds against a shared HMAC secret (D6) and against a verifier that obeys the envelope's algorithm label (D7n).

TABLE IV
ATTACK EXECUTIONS REMOVED AT EACH ABLATION STEP, WITH THE FAMILY RESPONSIBLE.

Step	Δ	Families newly blocked
D0→D1	90	duplicate-key malformed wire (30), signed over-limit (60)
D1→D2	300	unsigned forgery, bit flip, amount tamper, approval tamper, unknown key
D2→D3	60	expired mandate
D3→D4	60	exact replay (in-memory)
D4→D5	60	replay after restart
D5→D6	360	revoked, expired, not-yet-valid key; audience; policy downgrade; proposal swap
D6→D7	60	verifier-material leakage (Ed25519, pinned)

account for the next 420. The last 60 are the key-type issue of Section **??**.

For the full design, every family is rejected for the intended first-line reason (Table **??**), so the zero is not an accident of ordering. One subtlety: D0 already rejects the 30 truncated messages simply because its loose parser throws, which is why its count is 990 and not 1,020.

B. Crash recovery

Table **??** summarizes the 200 crash scenarios. With a non-idempotent exchange and blind retry, a crash after the effect is applied and a lost response after application both produce two effects, even though the local ledger never allowed a second *request*. Either an idempotent exchange or reconcile-first recovery yields exactly one effect at all five failure points. In every scenario the ledger ended in the CONFIRMED state,

TABLE V
DOMINANT REJECTION REASON PER ATTACK FAMILY IN THE FULL DESIGN.

Family	Rejection reason
unsigned forgery, bit flip, amount, approval tamper	signature
proposal swap	binding
exact replay, replay after restart	replay (ledger)
unknown key	key unknown
revoked, expired, not-yet-valid key	key status
expired mandate	expired
audience mismatch	audience
policy downgrade	policy version
signed over-limit	policy
verifier-material leakage	algorithm mismatch
malformed wire	malformed

TABLE VI
CRASH OUTCOMES OVER 10 REPETITIONS PER CELL. X = EXACTLY ONE EFFECT; DUP = TWO EFFECTS.

Exchange	Recovery	after reserve	after mark	after apply	after confirm	timeout after apply
non-idem.	blind retry	X	X	**DUP**	X	**DUP**
non-idem.	reconcile first	X	X	X	X	X
idem.	blind retry	X	X	X	X	X
idem.	reconcile first	X	X	X	X	X

and a client retry of the same wire after recovery added no effect (0 of 200). The label "exactly-once" is our harness verdict (one observed effect), not a claim that arbitrary remote systems offer such semantics.

C. Exhaustive model check

The model reproduces exactly this pattern (Table **??**). Only the non-idempotent, blind-retry configuration violates I1, with 156 reachable bad states, and it is also the only configuration with states from which confirmation with a single effect is unreachable (192 stuck states). The shortest counterexample has four steps: one gate reserves, marks dispatching, applies the effect but loses the response, and applies it again on retry. I2 and I3 hold in the other three configurations, which have 125 to 169 reachable states. The model is faithful only to its abstraction (two gates, bounded faults), so we use it as a consistency check on the harness, not as a proof about SQLite or a real exchange.

D. Concurrent duplicate submission

In all 30 trials of the process-local design (D4), the eight workers each applied the request, giving eight effects per trial. In all 30 trials of the full design (D7), exactly one worker won and exactly one effect was recorded; no internal errors occurred in either configuration. The gap is the practical reason a replay set held in memory is not a security control once a service runs more than one process. We note a confound: D4 differs from D7 in several ways at once (HMAC, no idempotent exchange, process-local replay), so this experiment shows that durable shared state *works*, not which of its features is responsible. A D5-versus-D7 pair on the same exchange would separate them and is listed as future work.

TABLE VII
MODEL-CHECK RESULTS (TWO RACING GATES, UP TO TWO CRASHES AND TWO LOST RESPONSES).

Exchange	Recovery	States	I1 viol.	I2 viol.	I3 stuck
non-idem.	blind retry	257	156	0	192
non-idem.	reconcile first	125	0	0	0
idem.	blind retry	169	0	0	0
idem.	reconcile first	125	0	0	0

TABLE VIII
ACCEPTANCE OF A REQUEST SIGNED BY A REVOKED KEY, BY CACHE TTL AND TIME AFTER REVOCATION.

TTL (s)	+1 s	+9 s	+119 s
0	rejected	rejected	rejected
5	**accepted**	rejected	rejected
60	**accepted**	**accepted**	rejected

E. Revocation staleness

Table **??** shows that caching creates a bounded stale-acceptance window controlled by the TTL. With no cache, every post-revocation request is rejected. With a 5 s TTL, the request one second after revocation is accepted from the warmed entry, and later ones are rejected. With a 60 s TTL, requests at 1 and 9 s are accepted and the one at 119 s is rejected. The window is measured from the last cache refresh, so in the worst case it is up to one TTL after revocation. High-risk deployments should use short TTLs, push invalidation, or an online status check for sensitive actions.

F. Performance

Table **??** gives end-to-end results, and Table **??** breaks down the components. The baseline is an in-memory path with no checks and is not security-equivalent to anything else, so we do not quote a "slowdown" factor against it. Three findings stand out. First, the signature scheme is not the bottleneck: Ed25519 verification costs about 143 μs at the median, roughly 3% of the 4.4 ms full-design median, and the HMAC legacy configuration is no faster end to end (20.2 ms against 18.5 ms for Ed25519 on the same storage settings). Second, storage settings dominate. Moving from per-call connections with rollback-journal mode and `FULL` sync to a persistent connection in WAL mode [**?**] raises throughput from about 52 to 198 requests/s, and relaxing to `NORMAL` sync raises it again to about 2,460 requests/s, with median latency falling to 0.32 ms. A single ledger reservation costs 2.9 ms under WAL+`FULL` and 28 μs under WAL+`NORMAL`, so the durability barrier is the cost. Third, key caching cuts a lookup from 11.9 μs to 0.56 μs but changes end-to-end throughput by only about 2% (the confidence intervals of the two medians do not overlap but differ by under 0.1 ms), at the price of the revocation window above.

G. Scripted-agent scenario

Table **??** shows executed fractions for the worst-case agent. Benign orders execute under every level, so the defenses do not break normal use. Over-limit and off-allowlist orders are

TABLE IX
END-TO-END LATENCY AND THROUGHPUT (5 REPEATS × 2,000 REQUESTS; MEAN ± S.D. OVER REPEATS FOR THROUGHPUT; 95% BOOTSTRAP INTERVAL FOR THE POOLED MEDIAN).

Configuration	Median (ms)	95% CI of median	p95 (ms)	p99 (ms)	Throughput (req/s)
D0 no checks (baseline)	0.007	[0.007, 0.007]	0.015	0.035	$82{,}515 \pm 1{,}816$
HMAC gate, legacy storage	20.198	[20.082, 20.306]	24.991	27.646	47.7 ± 1.4
Ed25519, DELETE+FULL	18.548	[18.466, 18.638]	23.824	26.432	51.9 ± 1.4
Ed25519, WAL+FULL	4.444	[4.423, 4.464]	9.517	12.570	198.2 ± 2.8
Ed25519, WAL+NORMAL	0.324	[0.322, 0.325]	0.396	0.627	$2{,}461.9 \pm 48.1$
Ed25519, WAL+FULL, 5 s key cache	4.347	[4.328, 4.364]	9.275	12.322	202.5 ± 2.1

TABLE X
COMPONENT MICROBENCHMARKS (MEDIAN / P99, μS).

Component	Median	p99
strict parse + schema	16.7	46.2
canonical signing input	8.4	20.6
HMAC sign / verify	4.4 / 13.2	13.2 / 31.2
Ed25519 sign / verify	45.0 / 143.1	81.2 / 198.7
ledger reserve, legacy	6,660	13,537
ledger reserve, WAL+FULL	2,936	6,422
ledger reserve, WAL+NORMAL	28.2	53.3
key lookup, new connection	172.5	311.0
key lookup, 1 conn / + 5 s cache	11.9 / 0.56	35.1 / 1.58

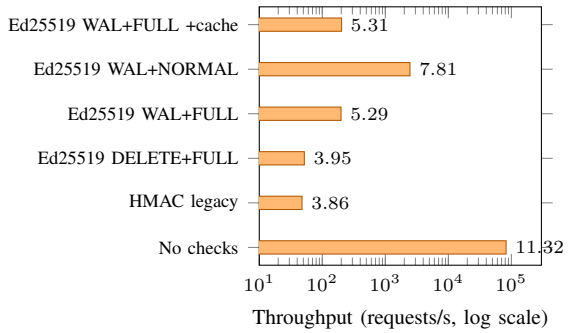


Fig. 4. Throughput by configuration. Storage settings, not the signature scheme, drive the gap between the gate rows.

TABLE XI
FRACTION OF CASES EXECUTED BY DEFENSE LEVEL (SCRIPTED WORST-CASE AGENT; THE AGENT FOLLOWED THE TARGET IN 100% OF INJECTED CASES BY CONSTRUCTION).

Scenario	n	L0	L1	L2	L3
benign	8	1.0	1.0	1.0	1.0
over-limit	24	1.0	0.0	0.0	0.0
off-allowlist	24	1.0	0.0	0.0	0.0
forged approval	24	1.0	1.0	1.0	**0.0**
in-policy manipulation	24	1.0	1.0	1.0	**1.0**

TABLE XII
EVIDENCE FOR EACH PROPERTY (E = EMPIRICAL IN THIS RUN; A = BY DESIGN ARGUMENT ONLY).

Property	Basis	Result
P1 invalid, no effect	E, attack matrix	0 of 1,020 executed (full design)
P2 single reservation	E, race	1 winner in 30 of 30 trials
P3 conflict detection	A	not measured in this run
P4 effect uniqueness	E, crash, model	1 effect where idempotent
P5 recovery progress	E, crash, model	all cases end CONFIRMED; 0 stuck states in safe configs
O6 residual risk	E, matrix, agent	60 of 60 in-policy executed

stopped by deterministic policy (L1) and by everything above it. The forged-approval goal shows why an in-band approval flag is not a control: it passes L1 and L2 because the order is inside policy and the sender simply claims approval, and only the full gate (L3) blocks it, since the forged envelope cannot verify. Finally, the in-policy manipulation executes under all four levels, including the full gate. This is the clearest empirical statement of O6: a signed, in-policy instruction produced by an agent that was steered is executed faithfully.

H. Summary of evidence by property

Table **??** ties each property and objective to the experiment that tests it and to the result, so a reader can see at a glance what is measured and what is argued.

VIII. DISCUSSION

Controls are complementary. Each failure mode in the corpus was closed by a different check, and the attribution in Table **??** shows that none of the controls could replace another. A signature does not say a message is current, intended for this service, or unused. Expiry shortens a mandate's life but does nothing about concurrent duplicates inside it. Durable replay tracking stops request reuse but cannot stop a valid mandate from being paired with a different proposal. The engineering consequence is to keep these as separate predicates with explicit rejection reasons and one test per predicate.

A signed decision is not a semantic guarantee. The residual family executes everywhere, and the agent scenario shows the same thing from the application side. If the upstream authorizer was manipulated or its policy is incomplete, the signature preserves the wrong decision faithfully. A trustworthy system therefore needs intent capture, policy governance, provenance of the inputs that shaped the proposal, risk limits that bound the damage of an allowed action, and human confirmation where the stakes justify it. The gate is the last deterministic line, not the first.

Exactly-once is an end-to-end property. The crash and model-check results agree: a local database cannot atomically commit an independent side effect. Which remedy is available depends on the receiver. If it accepts a durable idempotency key, use it. If it exposes status, reconcile before retrying. If it offers neither, an ambiguous timeout cannot be resolved safely

and should be escalated to a human or a settlement process rather than retried. In our simulator the exchange is idempotent by construction, so for real services this assumption must be verified, not inherited.

Operational trade-offs. The revocation window and the synchronization mode are both security parameters, not tuning knobs. `NORMAL` sync is about 12 times faster here than `FULL` but weakens what survives a power failure, and a longer cache TTL is cheaper but keeps a revoked key alive longer. A deployment should state its crash-consistency and revocation requirements first and measure on its own storage.

What this means for agent frameworks. Many agent frameworks let the model call a tool function directly and rely on prompt-level instructions to keep it in bounds. Our results suggest moving the decision out of the model's path. The proposal should be a plain data object, the authorization should arrive through a separate channel under a key the agent cannot reach, and the executor should be the only component that holds credentials for the downstream service. This does not stop an agent from being steered, as the in-policy family shows, but it turns a large class of failures (forgery, tampering, replay, cross-service reuse, stale approvals) from questions about model behavior into questions about deterministic checks that can be unit tested and audited. It also makes the interesting question sharper: if the checks are sound, which in-policy actions should an agent be allowed to propose at all, and who reviews them?

Relation to bearer tokens and capabilities. A signed mandate resembles a capability or a bearer token, and it inherits their familiar weakness: whoever holds the bytes can present them. What we add is the binding and the bookkeeping around it. The mandate is tied to one proposal digest, one audience, one policy version, and one request identifier, and the gate remembers what it has already accepted. Short expiry and one-time use together make a stolen mandate much less valuable than a stolen long-lived token, but they do not make it worthless inside its window. Sender-constrained credentials, where the presenter must also prove possession of a key, would close that gap, and we regard them as a natural extension rather than a competing design.

Cost of the controls. The extra work per request is small next to the work an agent already does. Verification is a single Ed25519 check and a few hash computations, and the durable reservation is one short transaction. The synchronization mode dominates the cost, which is why it has to be chosen deliberately. We did not measure network latency to a real exchange, and in a deployed system that latency, not the gate, will set end-to-end response time.

Deployment guidance that follows from the evidence. We would not call these recommendations validated for production, but the results point to a short list. (i) Pin the verification algorithm to trusted configuration and prefer an asymmetric scheme, so that leaking verifier material does not leak signing power. (ii) Keep policy, key-status, audience, version, and proposal binding as independent checks with their own tests. (iii) Put the replay reservation in storage shared by all workers and make rejected requests leave no state. (iv) Require an idempotency key or a status query from every downstream service that can cause an effect, and refuse to retry blindly where neither exists. (v) Choose the cache TTL and the synchronization mode from stated revocation and crash-consistency requirements, then measure. (vi) Treat semantic risk as a separate layer with its own limits and review, because none of the above addresses it.

IX. Limitations and Threats to Validity

Construct validity. The attack corpus was written alongside the implementation. It targets interpretable properties and is applied identically to every design, which supports comparison, but it is not a sample of real attacks. A count of 0 in 1,020 describes this corpus, and its Wilson bound describes sampling variation within it, not the probability that an unknown attack succeeds. An independently written generator, property-based fuzzing, and mutation testing would be the next step.

Internal validity. The ablation is cumulative, so steps that change several features at once cannot be read as isolated causal effects, and the race comparison has the confound noted above. Factorial or pairwise ablations (scheme, pinning, replay store, idempotency varied independently) would estimate interactions. Our run also contains no dedicated conflicting-identifier test, so P3 is a design property here and not a measured one.

External validity. The system is a local Python and SQLite prototype with a simulated exchange. It does not cover networks, remote services, replicated ledgers, hardware-backed keys, production identity, filesystem faults, or an adversarial host. Process termination at five chosen boundaries is not equivalent to every disk or network failure. The registry and policy are assumed trusted. The agent scenario is scripted and says nothing about how often a particular model is steered. Timings are single-machine, sequential, and environment-specific, and they should not be read as service capacity. The canonicalization is our own restricted scheme and is not shown to be interoperable. Finally, the related-work discussion is a positioning and not a systematic survey, and we have not compared against an existing agent-security system on a shared benchmark.

Ethics and responsible use. The experiments use synthetic keys, a simulated exchange, and generated requests only. No real service, account, or person was involved, and the attack families are standard integrity, replay, and confusion cases rather than new exploits. We describe weaknesses of the weaker designs, such as the verifier-leak and unlocked-race cases, because they motivate the stronger ones, and every weakness is a well-known class with a known remedy.

Scope of the claims. We do not claim that the full design is secure against every attacker, or that it prevents harm from a misled agent. We claim something narrower: on a stated corpus, with stated assumptions, each added control removes the failures it targets, and the remaining failure, the in-policy request, is exactly the one that no cryptographic check can address. That boundary is the main thing we want a reader to take away.

X. Conclusion and Future Work

We built and evaluated a layered authorization gate for tool-using agents. In the recorded synthetic evaluation the full

design blocked all 1,020 constructed attacks while accepting all 60 valid controls, a verifier that trusts the envelope's algorithm label failed exactly as the algorithm-confusion argument predicts, and a durable ledger gave one effect per trial where a process-local replay set gave eight. Crash injection and exhaustive model checking agreed that duplicate prevention after an ambiguous failure needs an idempotent receiver or reconciliation, and the performance and revocation measurements showed that storage durability and cache TTL, not the signature algorithm, set the cost and the staleness window.

None of this makes an agent safe. Authorization integrity, policy, context binding, replay protection, and effect recovery are separate controls, and none of them can tell whether an in-policy action reflects what the user actually wants. The next steps we consider most valuable are an independently generated attack suite, a D5-versus-D7 factorial ablation, multi-host and replicated-ledger failure schedules, validating the model against implementation traces, running the optional language-model agents with the same defense levels, and a head-to-head comparison with existing agent-security systems. Production readiness and general exactly-once behavior are not claimed.

ACKNOWLEDGMENT

The author received no external funding for this work.

REFERENCES

[1] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, Jan. 2017.
[2] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, "High-speed high-security signatures," *J. Cryptographic Eng.*, vol. 2, no. 2, 2012.
[3] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997.
[4] A. Rundgren, B. Jordan, and S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, June 2020.
[5] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, May 2015.
[6] Y. Sheffer, D. Hardt, and M. Jones, "JSON Web Token Best Current Practices," RFC 8725, Feb. 2020.
[7] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, and M. Lentczner, "Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud," in *Proc. NDSS*, 2014.
[8] R. M. Needham and M. D. Schroeder, "Using encryption for authentication in large networks of computers," *Commun. ACM*, vol. 21, no. 12, 1978.
[9] D. E. Denning and G. M. Sacco, "Timestamps in key distribution protocols," *Commun. ACM*, vol. 24, no. 8, 1981.
[10] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proc. CIDR*, 2007.
[11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.
[12] G. J. Holzmann, "The model checker SPIN," *IEEE Trans. Softw. Eng.*, vol. 23, no. 5, 1997.
[13] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.
[14] E. Debenedetti *et al.*, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.
[15] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of ACL*, 2024.
[16] E. Debenedetti *et al.*, "Defeating prompt injections by design," arXiv:2503.18813, 2025.
[17] OWASP GenAI Security Project, "LLM Top 10 for Large Language Model Applications," project documentation. [Online]. Available: https://genai.owasp.org/
[18] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, 1927.
[19] SQLite Consortium, "Write-Ahead Logging," SQLite documentation. [Online]. Available: https://www.sqlite.org/wal.html